GIT AND GITHUB CHALLENGE

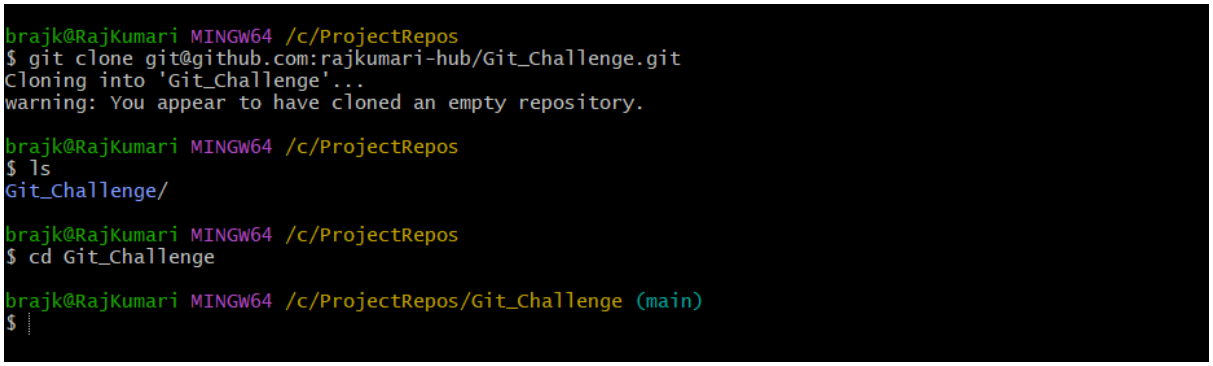
1.Resolve Merge Conflicts

- Create a merge conflict intentionally (two users editing the same line).
- Resolve the conflict and push the changes.

Step 1: Clone the Repository

Clone the GitHub repository to your local machine

$ git clone git@github.com:rajkumari-hub/Git_Challenge.git



Step 2: Create the First Branch

Create a new branch named branch1.

$ git checkout -b branch1

Create a file sample.txt by using

$ touch sample.txt or

$ vi sample.txt or

$ echo “add content”> sample.txt

$ git add .

$ git commit -m “ add sample.txt in branch1”

```
brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ git checkout -b branch1
Switched to a new branch 'branch1'

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (branch1)
$ git branch

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (branch1)
$
```

```
Welcome to git from branch1
~
```

Step 3: Create the Second Branch

Switch back to the main branch.

$ git checkout -b main

Create another branch named branch2.

$ git checkout -b branch2

$ git add .

$ git commit -m “add sample.txt in branch2”

```
brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (branch1)
$ echo "Welcome to git from branch1"> sample.txt

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (branch1)
$ ls
sample.txt

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (branch1)
$ git add .
warning: in the working copy of 'sample.txt', LF will be replaced by CRLF the next time Git touches it

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (branch1)
$ git commit -m "add sample.txt in branch1"
[branch1 (root-commit) 6f26c97] add sample.txt in branch1
 1 file changed, 1 insertion(+)
 create mode 100644 sample.txt

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (branch1)
$
```

```
Welcome to git from branch2
~
```

```
brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (branch2)
$ vi sample.txt

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (branch2)
$ ls
sample.txt

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (branch2)
$ git add .
warning: in the working copy of 'sample.txt', LF will be replaced by CRLF the next time Git touches it

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (branch2)
$ git commit -m "add sample.txt in branch2"
[branch2 e7c5952] add sample.txt in branch2
 1 file changed, 1 insertion(+), 1 deletion(-)

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (branch2)
```

Step 4: Merge Branch1 into Branch2

Make sure you are on branch2.

$ git merge branch1

```
brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (branch2)
$ git merge branch1
Auto-merging sample
CONFLICT (content): Merge conflict in sample
Automatic merge failed; fix conflicts and then commit the result.

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (branch2|MERGING)
$
```

Step 5: Resolve the conflict

Open sample You'll see:

```
<<<<<<< HEAD
Welcome to git from branch2
=======
Welcome to git from branch1
>>>>>>> branch1
~
~
```

Replace it with:

```
Welcome to git from branch1 and branch2
~
~
```

$ git add sample

$ git commit -m " resolved merge conflict"

```
brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (branch2|MERGING)
$ ls
sample

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (branch2|MERGING)
$ vi sample

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (branch2|MERGING)
$ git add sample

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (branch2|MERGING)
$ git commit -m "resolved merge conflict"
[branch2 b2d3e08] resolved merge conflict

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (branch2)
$
```

Step 6: Merge into main

$ git checkout main

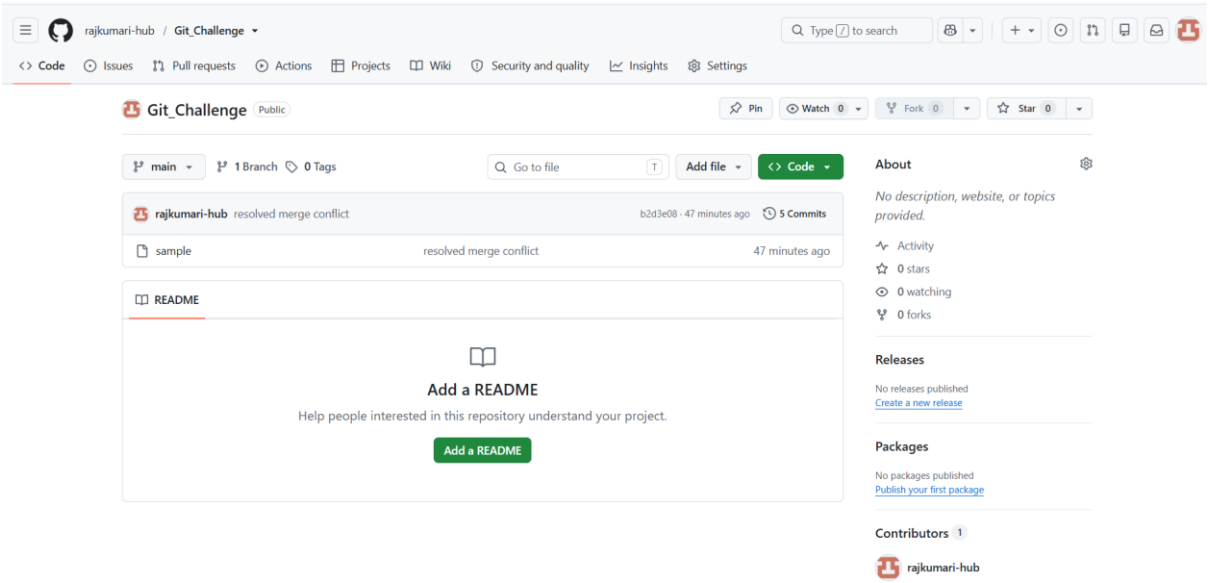
$ git merge branch2

Step 7: Push to GitHub

$ git push -u origin main

```
brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ ls
sample

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ cat sample

Welcome to git from branch1 and branch2

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ git push -u origin main
Enumerating objects: 14, done.
Counting objects: 100% (14/14), done.
Delta compression using up to 12 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (14/14), 1.12 KiB | 574.00 KiB/s, done.
Total 14 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), done.
To github.com:rajkumari-hub/Git_Challenge.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$
```

- Verify that the sample file has been pushed to the GitHub repository.



2. Recover Deleted Branch

Delete a local branch and then recover it using the reflog.

Step 1: Open Git Bash

Navigate to your Git repository.

$ cd /c/ProjectRepos/Git_Challenge

Step 2: Create a New Branch

Create a branch named feature.

$ git checkout -b feature

$ git branch

Step 3: Create a File

Create a sample file using vi editor

$ vi devops

Step 4: Add the File

$ git add devops

Step 5: Commit the Changes

$ git commit -m "Added future file"

```
brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ git checkout -b feature
Switched to a new branch 'feature'

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (feature)
$ vi devops

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (feature)
$ ls
devops  sample

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (feature)
$ git add devops
warning: in the working copy of 'devops', LF will be replaced by CRLF the next time Git touches it

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (feature)
$ git commit -m "added devops file"
[feature 92aa8b4] added devops file
 1 file changed, 1 insertion(+)
 create mode 100644 devops

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (feature)
$
```

Step 6: Switch Back to Main Branch

$ git checkout main

Step 7: Delete the Local Branch

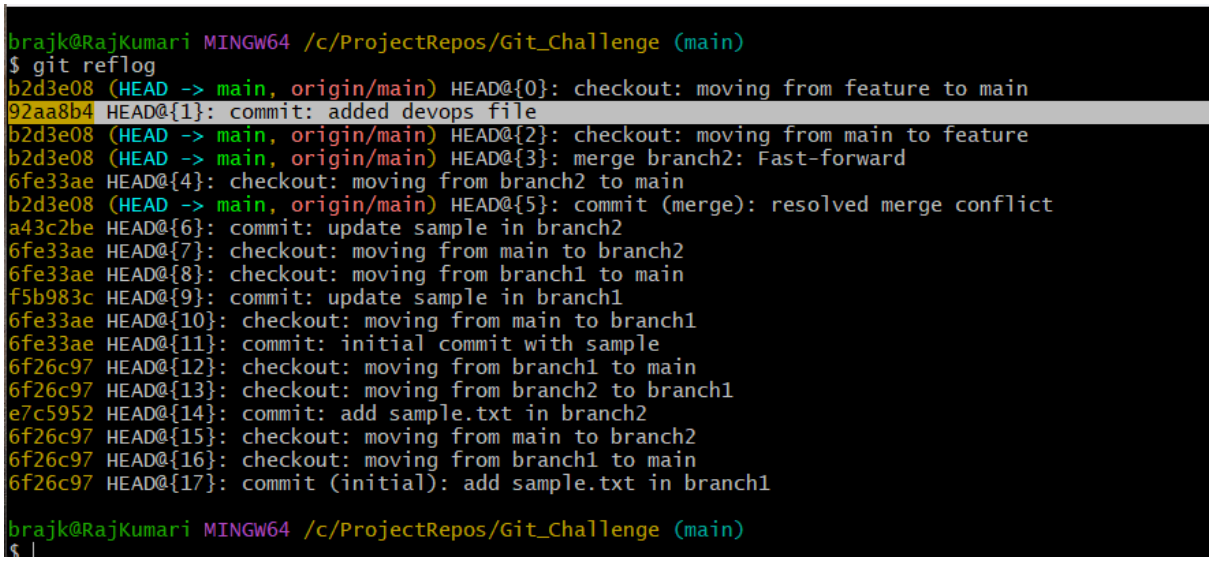
Delete the branch.

$ git branch -d feature

Step 8: View the Reflog

Display the reflog.

$ git reflog



Step 9: Recover the Deleted Branch

Create the branch again using the commit hash.

$ git checkout -b feature 92aa8b4

Step 10: Verify the Recovery

Check the branches.

$ git branch

Step 11: Verify the Commit History

git log --oneline

Step 12: Verify the File

$ls

```
brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ git checkout -b feature 92aa8b4
Switched to a new branch 'feature'

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (feature)
$ git branch
* feature
  main

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (feature)
$ git log --oneline
92aa8b4 (HEAD -> feature) added devops file
b2d3e08 (origin/main, main) resolved merge conflict
a43c2be update sample in branch2
f5b983c update sample in branch1
6fe33ae initial commit with sample
6f26c97 add sample.txt in branch1

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (feature)
$ ls
devops  sample

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (feature)
$ cat devops
welcome to devops

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (feature)
$
```

3. Undo Wrong Push

- Push a wrong commit to GitHub, then undo it without losing history.

Step 1: Create a Sample File

Create a file that will represent the "wrong" change.

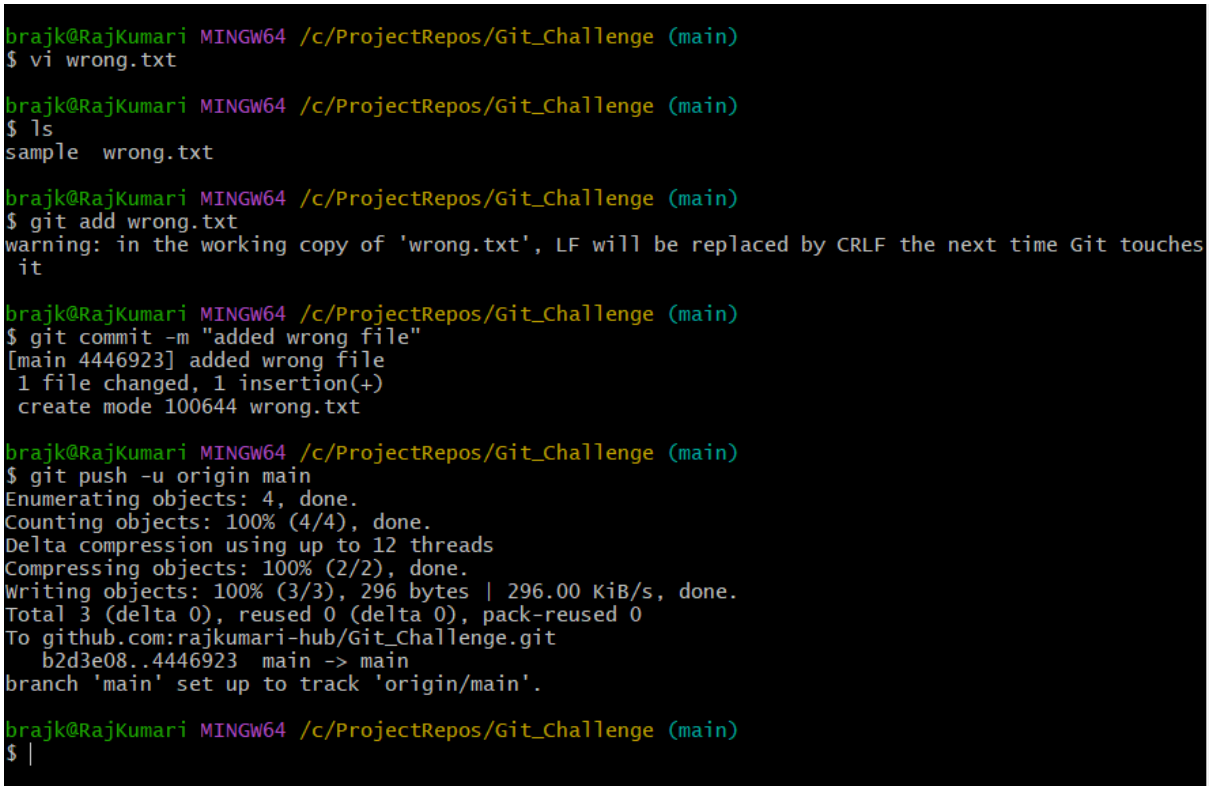
$ vi wrong.txt

$ git add wrong.txt

$ git commit -m "Added wrong file"

Step 2: Push the Commit to GitHub

$ git push origin main



Step 3: View the Commit History

$ git log --oneline

Step 4: Undo the Wrong Commit

Create a new commit that reverses the changes.

$ git revert HEAD

If your editor opens:

1. Keep the default commit message.
2. Save and close the editor

Step 5: Verify the History

$ git log --oneline

```
brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ git log --oneline
4446923 (HEAD -> main, origin/main) added wrong file
b2d3e08 resolved merge conflict
a43c2be update sample in branch2
f5b983c update sample in branch1
6fe33ae initial commit with sample
6f26c97 add sample.txt in branch1

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ git revert HEAD
[main 4dbcd3c] Revert "added wrong file"
 1 file changed, 1 deletion(-)
 delete mode 100644 wrong.txt

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ git log --oneline
4dbcd3c (HEAD -> main) Revert "added wrong file"
4446923 (origin/main) added wrong file
b2d3e08 resolved merge conflict
a43c2be update sample in branch2
f5b983c update sample in branch1
6fe33ae initial commit with sample
6f26c97 add sample.txt in branch1

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$
```

```
Revert "added wrong file"

This reverts commit 44469231f4b27d129cdcaa47d0f6db49cce4084e.

# Please enter the commit message for your changes. Lines starting
# with '#' will be ignored, and an empty message aborts the commit.
#
# On branch main
# Your branch is up to date with 'origin/main'.
#
# Changes to be committed:
#       deleted:    wrong.txt
#
~
~
```

Step 6: Push the Revert Commit

$ git push origin main

Step 7: Verify the Working Directory

$ ls

The wrong.txt file will no longer be present because the revert removed it.

$ git status

```
brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ git push -u origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 12 threads
Compressing objects: 100% (1/1), done.
Writing objects: 100% (2/2), 259 bytes | 259.00 KiB/s, done.
Total 2 (delta 0), reused 0 (delta 0), pack-reused 0
To github.com:rajkumari-hub/Git_Challenge.git
   4446923..4dbcd3c  main -> main
branch 'main' set up to track 'origin/main'.

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ ls
sample

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$
```

4. Amend a Commit

- Make a commit, then add a missing file to it using $ git commit --amend.

Step 1: Create the First File

$ vi git.txt

$ git add git.txt

$ git commit -m “added git file”

```
brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ vi git.txt

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ ls
git.txt  sample

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ git add git.txt
warning: in the working copy of 'git.txt', LF will be replaced by CRLF the next time Git touches i
t

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ git commit -m "Add git file"
[main d3c8016] Add git file
 1 file changed, 1 insertion(+)
 create mode 100644 git.txt

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
```

Step2: Realize You Forgot Another File

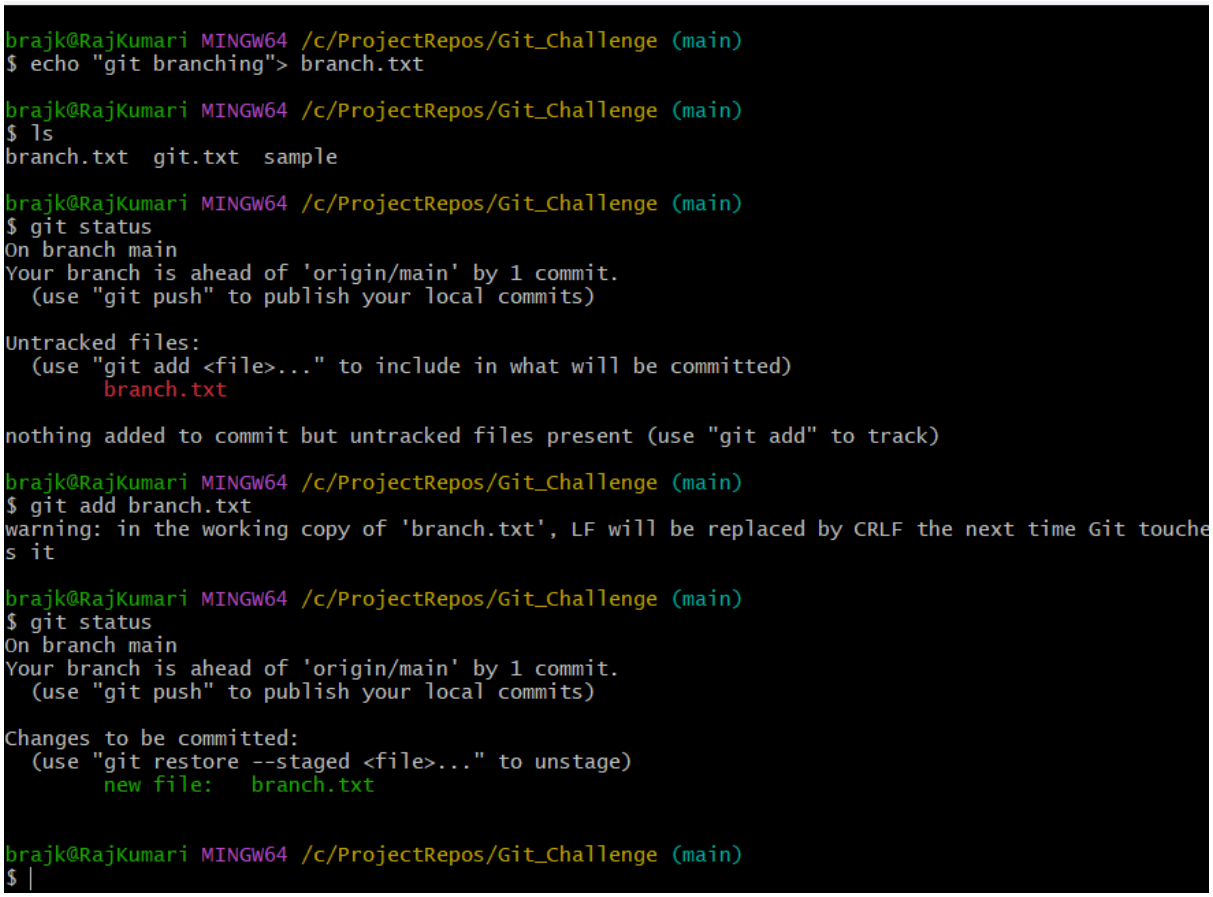
Create another file.

$ echo "Git Branching" > branch.txt

$ git status

$ git add .

$ git staus



Step 3: Amend the Previous Commit

Run: $ git commit --amend

Your default editor will open.

You'll see the previous commit message

```
Add git file

# Please enter the commit message for your changes. Lines starting
# with '#' will be ignored, and an empty message aborts the commit.
#
# Date:      Fri Jul 10 10:53:02 2026 +0530
#
# On branch main
# Your branch is ahead of 'origin/main' by 1 commit.
#   (use "git push" to publish your local commits)
#
# Changes to be committed:
#       new file:   branch.txt
#       new file:   git.txt
#
~
```

```
brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ git commit --amend
[main 3ab3de9] Add git file
 Date: Fri Jul 10 10:53:02 2026 +0530
 2 files changed, 2 insertions(+)
 create mode 100644 branch.txt
 create mode 100644 git.txt

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ git log --oneline
3ab3de9 (HEAD -> main) Add git file
4dbcd3c (origin/main) Revert "added wrong file"
4446923 added wrong file
b2d3e08 resolved merge conflict
a43c2be update sample in branch2
f5b983c update sample in branch1
6fe33ae initial commit with sample
6f26c97 add sample.txt in branch1

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ ls
branch.txt  git.txt  sample

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ git push origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (4/4), 347 bytes | 347.00 KiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0
To github.com:rajkumari-hub/Git_Challenge.git
   4dbcd3c..3ab3de9  main -> main

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$
```

5. Cherry-pick a Commit

- Take a specific commit from one branch and apply it to another branch.

Step1: Create a New Branch

Create a branch named **feature**.

$ git checkout -b feature

Create a new file using vi editor

$ vi cherrt.txt

$ git add .

$ git commit -m “added cherry pick”

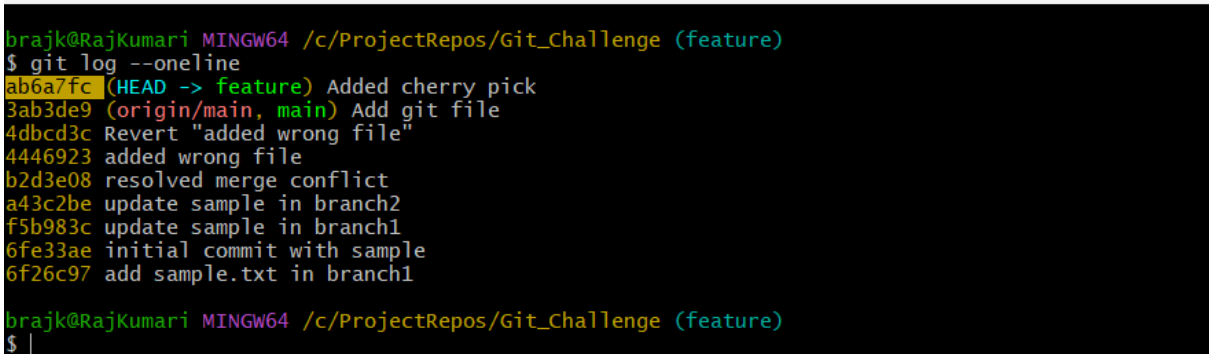
```
brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ git branch
* main

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ git checkout -b feature
Switched to a new branch 'feature'

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (feature)
$ echo "Cherry Pick Example" > cherry.txt

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (feature)
$ ls
branch.txt  cherry.txt  git.txt  sample

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (feature)
$ git add cherry.txt
warning: in the working copy of 'cherry.txt', LF will be replaced by CRLF the next time Git touche
s it

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (feature)
$ git commit -m "Added cherry pick"
[feature ab6a7fc] Added cherry pick
 1 file changed, 1 insertion(+)
 create mode 100644 cherry.txt

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (feature)
$
```

Step2: View the commit hash

$ git log - -oneline

Here, copy the commit id/hash of cherry pick

Ex: ab6a7fc



Step 3: Switch to Main Branch

$ git checkout main

Step 4: Cherry-pick the Commit

Use the commit hash copied earlier.

$ git cherry-pick ab6a7fc

- The **message is the same**
- The **commit hash is different**

Git created a **new commit** with the same changes.

Step 5: Verify the Commit History

$ git log --oneline

Step 6: Verify the File

$ ls

cherry.txt

Step 7: Check Both Branches

$git branch

```
brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (feature)
$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ ls
branch.txt  git.txt  sample

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ git cherry-pick ab6a7fc
[main 046d08e] Added cherry pick
 Date: Fri Jul 10 11:07:13 2026 +0530
 1 file changed, 1 insertion(+)
 create mode 100644 cherry.txt

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ ls
branch.txt  cherry.txt  git.txt  sample

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ git branch
  feature
* main

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$
```

```
brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ git log --oneline
046d08e (HEAD -> main) Added cherry pick
3ab3de9 (origin/main) Add git file
4dbcd3c Revert "added wrong file"
4446923 added wrong file
b2d3e08 resolved merge conflict
a43c2be update sample in branch2
f5b983c update sample in branch1
6fe33ae initial commit with sample
6f26c97 add sample.txt in branch1

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$
```

6. Interactive Rebase

- Reorder and squash multiple commits into a single clean commit.

Step 1: Create the First File

$ vi student

$ git add .

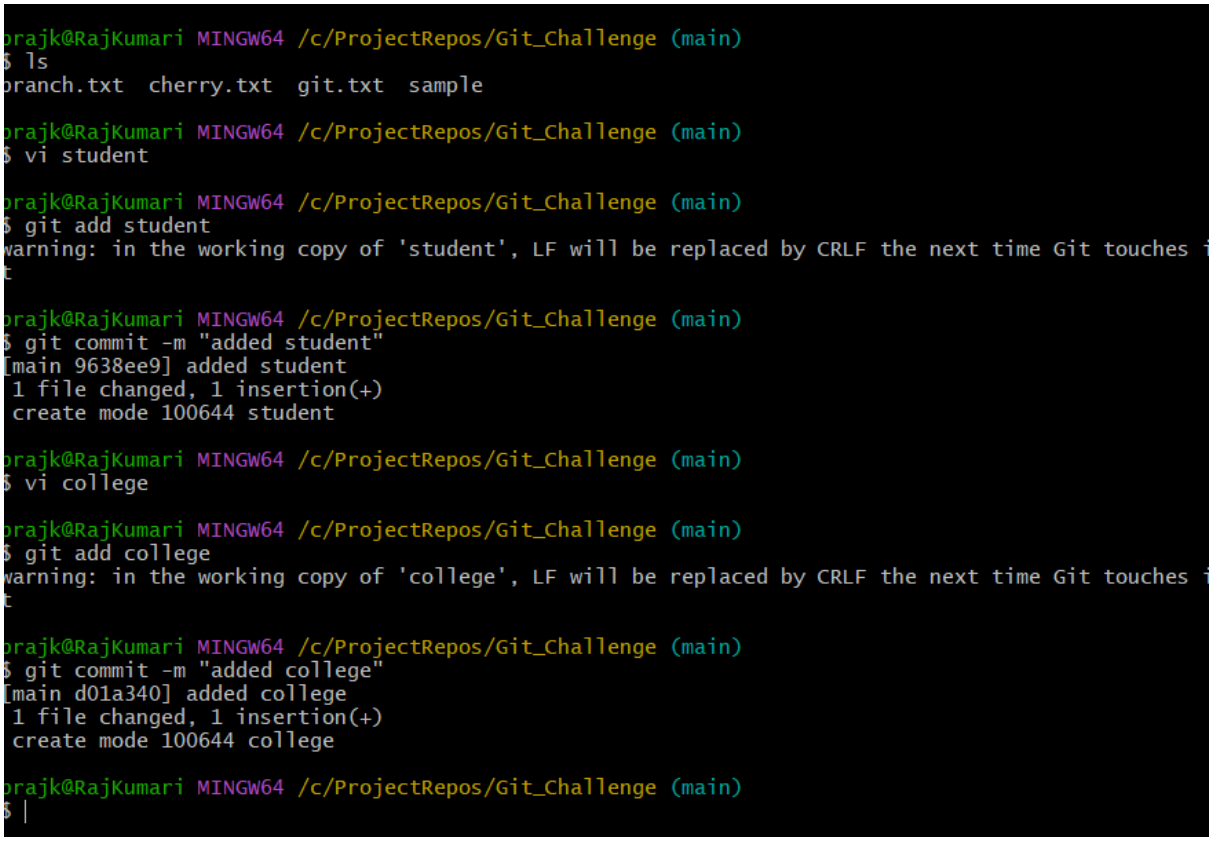
$ git commit -m “added student”

Step2: Create the second file

$ vi college

$ git add .

$ git commit -m “added college”



Step3: Verify the Commit History

$ git log --oneline

Step 4: Start Interactive Rebase

We want to modify the last **4 commits**.

$ git rebase -i HEAD~4

```
brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ git log --oneline
4dc8e4b (HEAD -> main) added rules
f26992e added college
88419fc added student
2eadd1b add student college rules files
e2dd6d3 Added cherry pick
3ab3de9 (origin/main) Add git file
4dbcd3c Revert "added wrong file"
4446923 added wrong file
b2d3e08 resolved merge conflict
a43c2be update sample in branch2
f5b983c update sample in branch1
6fe33ae initial commit with sample
6f26c97 add sample.txt in branch1

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$
```

```
pick 2eadd1b add student college rules files
squash 88419fc added student
squash f26992e added college
squash 4dc8e4b added rules

# Rebase e2dd6d3..4dc8e4b onto e2dd6d3 (4 commands)
#
# Commands:
# p, pick <commit> = use commit
# r, reword <commit> = use commit, but edit the commit message
# e, edit <commit> = use commit, but stop for amending
# s, squash <commit> = use commit, but meld into previous commit
# f, fixup [-C | -c] <commit> = like "squash" but keep only the previous
#                    commit's log message, unless -C is used, in which case
#                    keep only this commit's message; -c is same as -C but
#                    opens the editor
# x, exec <command> = run command (the rest of the line) using shell
# b, break = stop here (continue rebase later with 'git rebase --continue')
# d, drop <commit> = remove commit
# l, label <label> = label current HEAD with a name
# t, reset <label> = reset HEAD to a label
# m, merge [-C <commit> | -c <commit>] <label> [# <oneline>]
```

Step 4: Edit the Commit Message

Git opens another editor with: save and close the editor

```
# This is a combination of 4 commits.
# This is the 1st commit message:

add student college rules files in one commit

# This is the commit message #2:



# This is the commit message #3:



# This is the commit message #4:



# Please enter the commit message for your changes. Lines starting
# with '#' will be ignored, and an empty message aborts the commit.
#
# Date:      Fri Jul 10 12:13:21 2026 +0530
#
# interactive rebase in progress; onto e2dd6d3
```

```
brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ git rebase -i HEAD~4
[detached HEAD ef412d9] add student college rules files in one commit
 Date: Fri Jul 10 12:13:21 2026 +0530
 3 files changed, 3 insertions(+), 2 deletions(-)
 create mode 100644 rules
Successfully rebased and updated refs/heads/main.

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ git log --oneline
ef412d9 (HEAD -> main) add student college rules files in one commit
e2dd6d3 Added cherry pick
3ab3de9 (origin/main) Add git file
4dbcd3c Revert "added wrong file"
4446923 added wrong file
b2d3e08 resolved merge conflict
a43c2be update sample in branch2
f5b983c update sample in branch1
6fe33ae initial commit with sample
6f26c97 add sample.txt in branch1

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$
```

```
brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ git log
commit ef412d962b9e8c17d9609bc8783cf15ded93f378 (HEAD -> main)
Author: rajkumari <brajkumari7675@gmail.com>
Date:   Fri Jul 10 12:13:21 2026 +0530

    add student college rules files in one commit
```

7. Tagging & Release

- Create a version tag (v1.0), push it to GitHub, then delete and restore it.

Step 1: Create an Annotated Tag

$ git tag -a v1.0 -m "Version 1.0 Release"

Explanation:

- git tag → Creates a tag.
- -a → Creates an annotated tag.
- v1.0 → Tag name.
- -m → Tag message.

Step2: Verify tag and tag details

$ git tag

$ git show v1.0

```
$ git tag -a v1.0 -m "Version 1.0 Release"

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ git tag
v1.0

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ git show v1.0
tag v1.0
Tagger: rajkumari <brajkumari7675@gmail.com>
Date:   Fri Jul 10 12:39:01 2026 +0530

Version 1.0 Release

commit ef412d962b9e8c17d9609bc8783cf15ded93f378 (HEAD -> main, tag: v1.0)
Author: rajkumari <brajkumari7675@gmail.com>
Date:   Fri Jul 10 12:13:21 2026 +0530

    add student college rules files in one commit

diff --git a/college b/college
index c1f7355..b8c4780 100644
--- a/college
+++ b/college
@@ -1 +1 @@
-My college is good
+college
diff --git a/rules b/rules
new file mode 100644
index 0000000..51f1f8a
--- /dev/null
+++ b/rules
@@ -0,0 +1 @@
+rulesss
diff --git a/student b/student
index fc89568..ca0e242 100644
--- a/student
+++ b/student
@@ -1 +1 @@
-Student life is very nice
+student
```

Step3: Push the Tag to GitHub

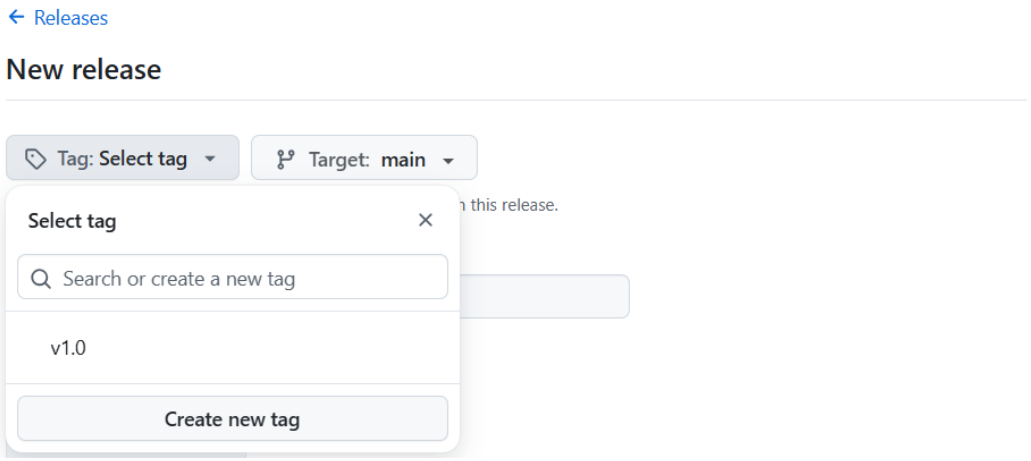
$ git push origin v1.0

```
brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ git push origin v1.0
Enumerating objects: 12, done.
Counting objects: 100% (12/12), done.
Delta compression using up to 12 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (11/11), 968 bytes | 484.00 KiB/s, done.
Total 11 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), done.
To github.com:rajkumari-hub/Git_Challenge.git
 * [new tag]         v1.0 -> v1.0

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$
```

Step 4: Verify the Tag on GitHub

1. Open your GitHub repository.

2. Click **Releases** or **Tags**.



Step 5: Delete the Local Tag

$ git tag -d v1.0

Verify:

$ git tag

No tags will be listed.

Step 6: Delete the Remote Tag (GitHub)

$ git push origin --delete v1.0

```
brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ git tag -d v1.0
Deleted tag 'v1.0' (was 2027e52)

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ git tag

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ git push origin --delete v1.0
To github.com:rajkumari-hub/Git_Challenge.git
 - [deleted]         v1.0

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$
```

Step 7: Restore the Deleted Tag

Find the commit you want to tag again.

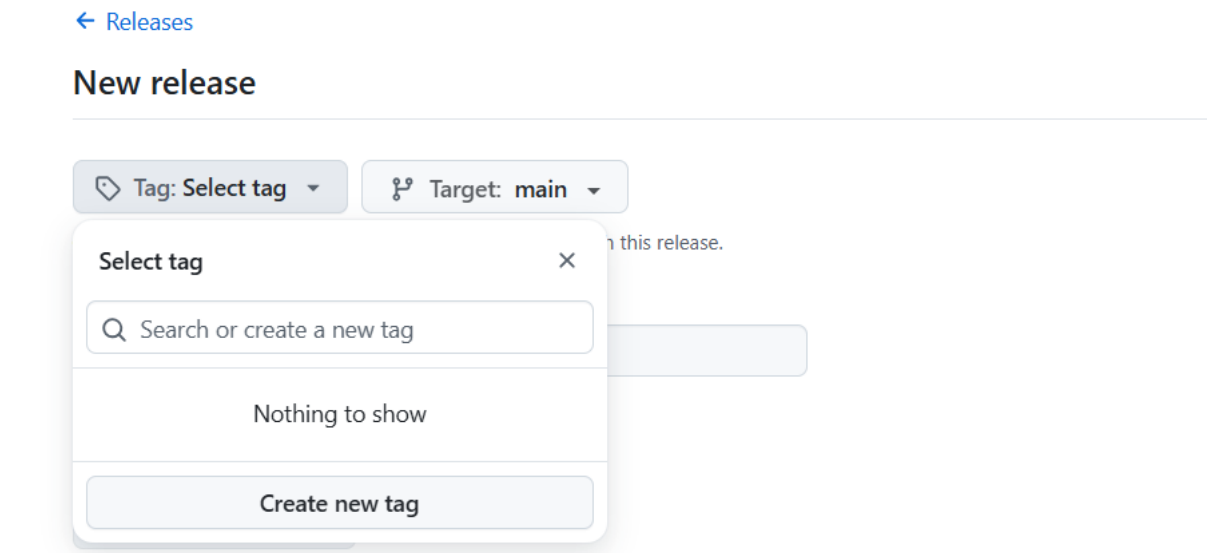
$ git log –oneline

Restore the tag using commit hash

$ git tag -a v1.0 ef412d9 -m “Version 1.0 Release”

$ git tag

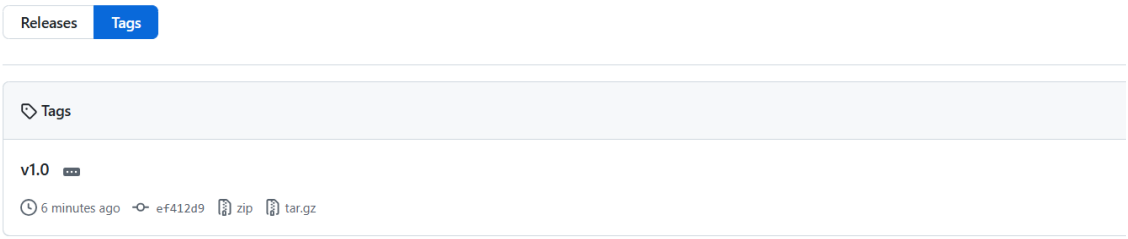
```
brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ git log --oneline
ef412d9 (HEAD -> main) add student college rules files in one commit
e2dd6d3 Added cherry pick
3ab3de9 (origin/main) Add git file
4dbcd3c Revert "added wrong file"
4446923 added wrong file
b2d3e08 resolved merge conflict
a43c2be update sample in branch2
f5b983c update sample in branch1
6fe33ae initial commit with sample
6f26c97 add sample.txt in branch1

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ git tag -a v1.0 ef412d9 -m "Version 1.0 Release"

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ git tag
v1.0

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$
```



Step 8: Push the Restored Tag

$ git push origin v1.0

```
brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ git push origin v1.0
Enumerating objects: 12, done.
Counting objects: 100% (12/12), done.
Delta compression using up to 12 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (11/11), 968 bytes | 968.00 KiB/s, done.
Total 11 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), done.
To github.com:rajkumari-hub/Git_Challenge.git
 * [new tag]         v1.0 -> v1.0

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$
```

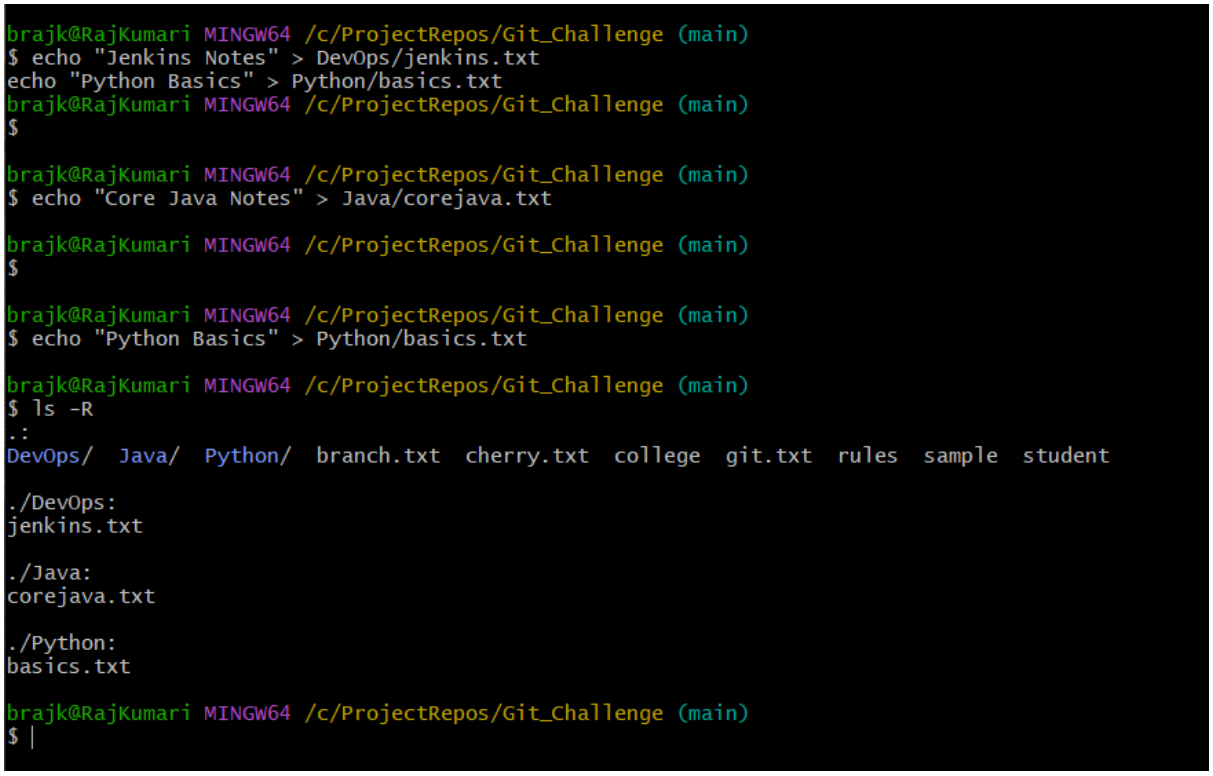


8. Clone with Sparse Checkout

- Clone only a subdirectory of a repo using sparse checkout.

Sparse checkout in **cone mode** is designed to work with **directories**.

Before performing Sparse Checkout we need to create a direcories



Step 1: Move to the Parent Directory

$ cd ..

Step 2: Clone the Repository Without Checking Out Files

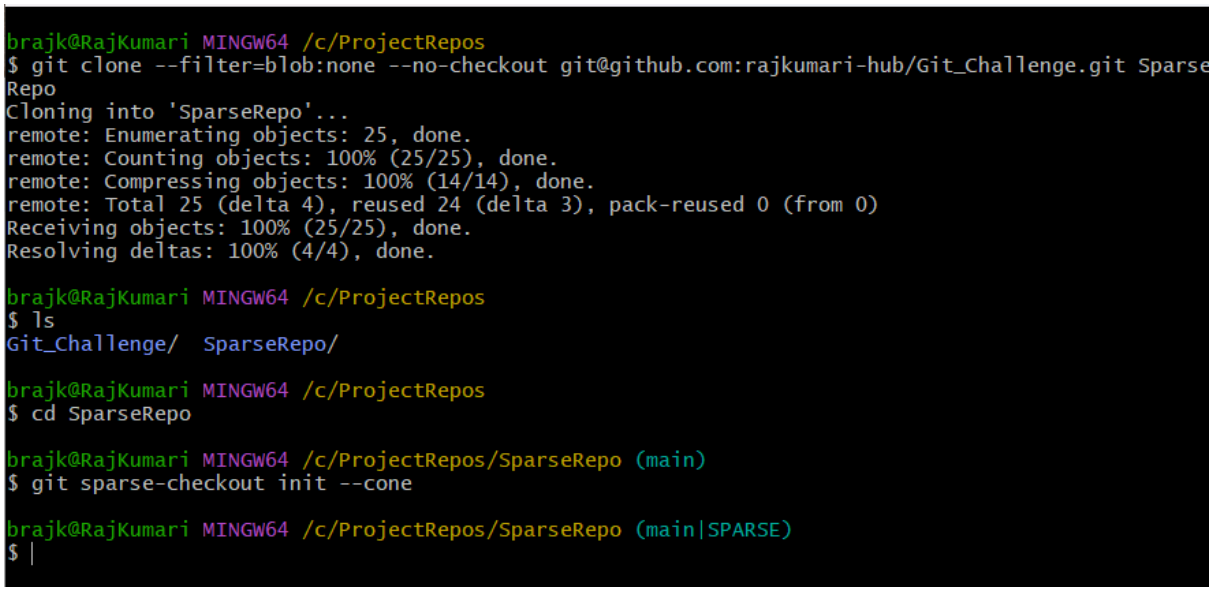
git clone --filter=blob:none --no-checkout https://github.com/<username>/Git_Challenge.git SparseRepo

Step 3: Navigate to the Cloned Repository

$ cd SparseRepo

Step 4: Initialize Sparse Checkout

$ git sparse-checkout init - -cone



Step 5: Checkout Only the Required Directory

$ git sparse-checkout set DevOps

Step 6: Checkout the Main Branch

$ git checkout main

Step 7: Verify the Repository Contents

$ ls

Step 8: Verify the Selected Directory

$ ls DevOps

Step 9: (Optional) Add Another Directory

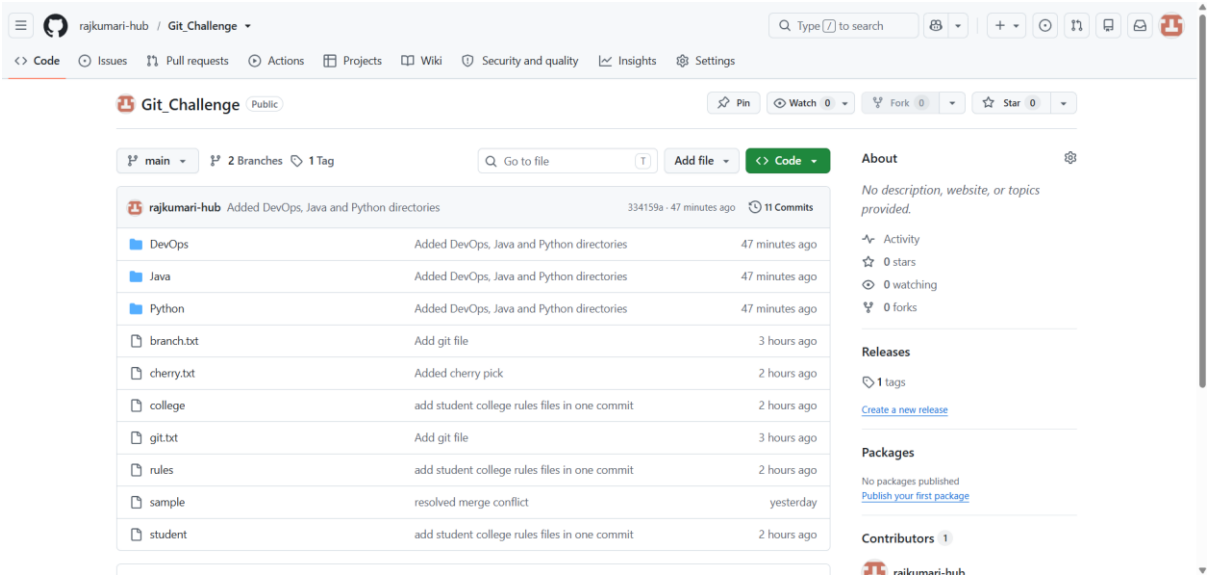
$ git sparse-checkout add Java

Step 10: (Optional) Disable Sparse Checkout

$ git sparse-checkout disable

```
brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ git add .

git push origin mainwarning: in the working copy of 'DevOps/jenkins.txt', LF will be replaced by
RLF the next time Git touches it
warning: in the working copy of 'Java/corejava.txt', LF will be replaced by CRLF the next time Gi
 touches it
warning: in the working copy of 'Python/basics.txt', LF will be replaced by CRLF the next time Gi
 touches it

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ git commit -m "Added DevOps, Java and Python directories"
[main 334159a] Added DevOps, Java and Python directories
 3 files changed, 3 insertions(+)
 create mode 100644 DevOps/jenkins.txt
 create mode 100644 Java/corejava.txt
 create mode 100644 Python/basics.txt

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ git push origin main
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (8/8), 562 bytes | 562.00 KiB/s, done.
Total 8 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:rajkumari-hub/Git_Challenge.git
   3ab3de9..334159a  main -> main

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$
```

```
brajk@RajKumari MINGW64 /c/ProjectRepos/SparseRepo (main|SPARSE)
$ git sparse-checkout set DevOps

brajk@RajKumari MINGW64 /c/ProjectRepos/SparseRepo (main|SPARSE)
$ git checkout main
remote: Enumerating objects: 8, done.
remote: Counting objects: 100% (8/8), done.
remote: Total 8 (delta 0), reused 8 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (8/8), 232 bytes | 232.00 KiB/s, done.
Updating files: 100% (8/8), done.
Already on 'main'
Your branch is up to date with 'origin/main'.

brajk@RajKumari MINGW64 /c/ProjectRepos/SparseRepo (main|SPARSE)
$ ls
DevOps/  branch.txt  cherry.txt  college  git.txt  rules  sample  student

brajk@RajKumari MINGW64 /c/ProjectRepos/SparseRepo (main|SPARSE)
$ ls DevOps
jenkins.txt

brajk@RajKumari MINGW64 /c/ProjectRepos/SparseRepo (main|SPARSE)
$
```



9. Reset vs Revert Challenge

- Demonstrate the difference between git reset –hard and git revert in a repo.

Step1: Create a Sample File

$ echo "Reset Example" > reset.txt

Step 2: Stage the File

$ git add reset.txt

Step 3: Commit the Changes

$ git commit -m "Added reset example"

Step 4: View Commit History

$ git log --oneline

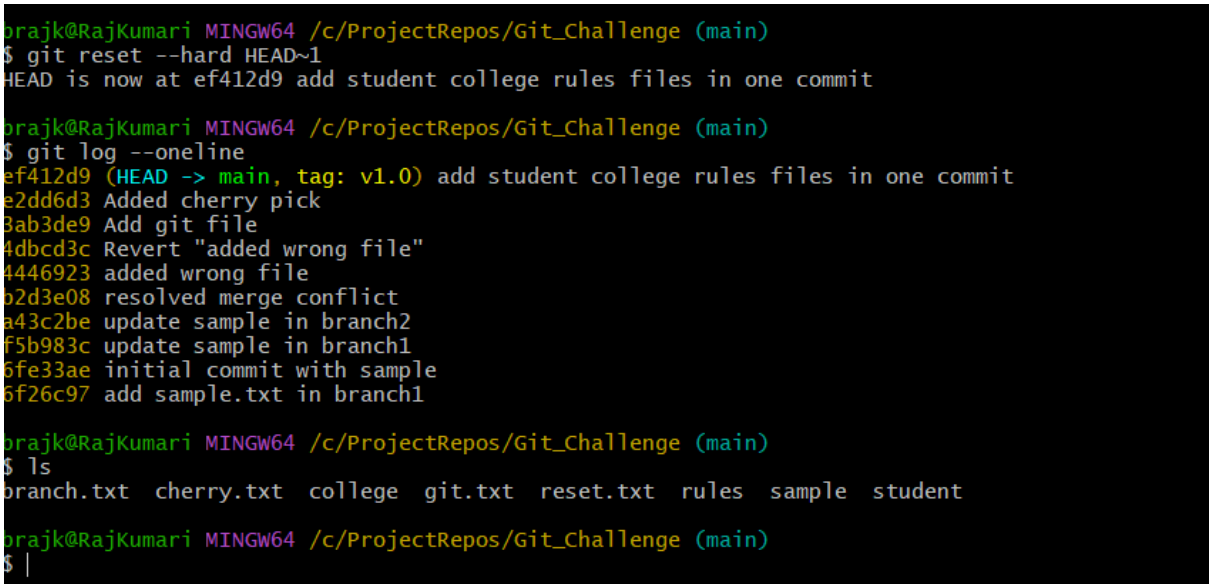
```
brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ ls
DevOps/  Java/  Python/  branch.txt  cherry.txt  college  git.txt  rules  sample  student

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ echo "Reset Example" > reset.txt

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ echo "Reset Example" > reset.txt

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ git commit -m "Added reset example"
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        reset.txt

nothing added to commit but untracked files present (use "git add" to track)

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ git log --oneline
334159a (HEAD -> main, origin/main) Added DevOps, Java and Python directories
ef412d9 (tag: v1.0) add student college rules files in one commit
e2dd6d3 Added cherry pick
3ab3de9 Add git file
4dbcd3c Revert "added wrong file"
4446923 added wrong file
b2d3e08 resolved merge conflict
a43c2be update sample in branch2
f5b983c update sample in branch1
6fe33ae initial commit with sample
6f26c97 add sample.txt in branch1

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$
```

Step5: Reset to the Previous Commit

$ git reset --hard HEAD~1

Step 6: Verify the Commit History

$ git log --oneline



Revert

Step1: Create Another Sample File

$ echo "Revert Example" > revert.txt

Step 2: Stage the File

$ git add revert.txt

Step 3: Commit the Changes

$ git commit -m "Added revert example"

Step 4: View Commit History

$ git log --oneline

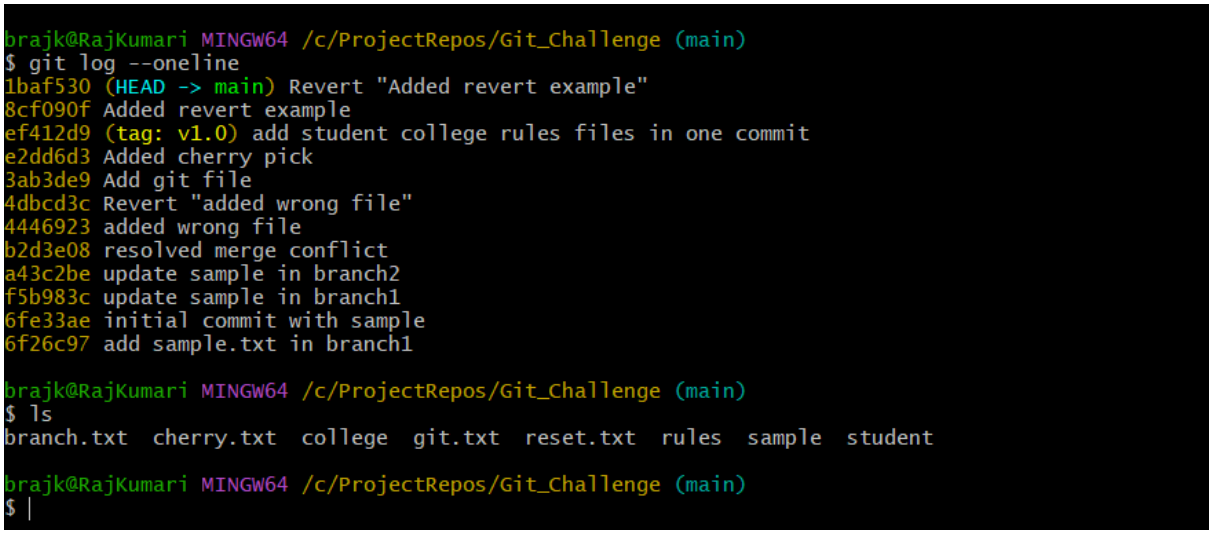
Step 5: Revert the Latest Commit

$ git revert HEAD

```
prajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ echo "Revert Example" > revert.txt

prajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ git add revert.txt
warning: in the working copy of 'revert.txt', LF will be replaced by CRLF the next time Git touche
s it

prajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ git commit -m "Added revert example"
[main 8cf090f] Added revert example
 1 file changed, 1 insertion(+)
 create mode 100644 revert.txt

prajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ git log --oneline
8cf090f (HEAD -> main) Added revert example
ef412d9 (tag: v1.0) add student college rules files in one commit
e2dd6d3 Added cherry pick
3ab3de9 Add git file
4dbcd3c Revert "added wrong file"
4446923 added wrong file
b2d3e08 resolved merge conflict
a43c2be update sample in branch2
f5b983c update sample in branch1
6fe33ae initial commit with sample
6f26c97 add sample.txt in branch1

prajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ git revert HEAD
[main 1baf530] Revert "Added revert example"
 1 file changed, 1 deletion(-)
 delete mode 100644 revert.txt

prajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$
```

```
Revert "Added revert example"

This reverts commit 8cf090f5c86c8228b279bf7c1fdb1a9c9a4e12f3.

# Please enter the commit message for your changes. Lines starting
# with '#' will be ignored, and an empty message aborts the commit.
#
# On branch main
# Your branch and 'origin/main' have diverged,
# and have 1 and 1 different commits each, respectively.
#   (use "git pull" to merge the remote branch into yours)
#
# Changes to be committed:
#       deleted:    revert.txt
#
# Untracked files:
#       reset.txt
#
```

revert.txt will no longer exist, but the commit history will contain both the original commit and the revert commit.



10. Detached HEAD Challenge

- Checkout a specific commit (detached HEAD state) and create a new branch from it.

Step1: View the Commit History

$ git log - -oneline

Copy the commit hash you want to check out.

Step 2: Checkout the Specific Commit

$ git checkout f26992e

```
brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ git log --oneline
1baf530 (HEAD -> main) Revert "Added revert example"
8cf090f Added revert example
ef412d9 (tag: v1.0) add student college rules files in one commit
e2dd6d3 Added cherry pick
3ab3de9 Add git file
4dbcd3c Revert "added wrong file"
4446923 added wrong file
b2d3e08 resolved merge conflict
a43c2be update sample in branch2
f5b983c update sample in branch1
6fe33ae initial commit with sample
6f26c97 add sample.txt in branch1

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ git checkout 3ab3de9
Note: switching to '3ab3de9'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

  git switch -c <new-branch-name>

Or undo this operation with:

  git switch -

Turn off this advice by setting config variable advice.detachedHead to false

HEAD is now at 3ab3de9 Add git file

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge ((3ab3de9...))
$
```

Step 3: Verify the Detached HEAD State

$ git status

```
brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge ((3ab3de9...))
$ git status
HEAD detached at 3ab3de9
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        reset.txt

nothing added to commit but untracked files present (use "git add" to track)

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge ((3ab3de9...))
$
```

Step 4: Create a New Branch from the Detached HEAD

$ git checkout -b detached-branch

Step 5: Verify the Branch

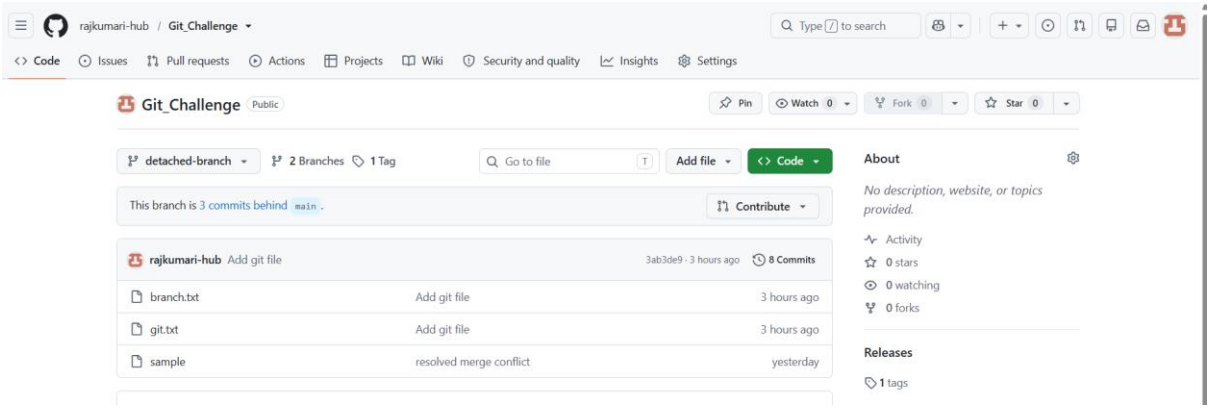
$ git branch

```
brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge ((3ab3de9...))
$ git checkout -b detached-branch
Switched to a new branch 'detached-branch'

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (detached-branch)
$ git branch
* detached-branch
  feature
  main

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (detached-branch)
$ git log --oneline
3ab3de9 (HEAD -> detached-branch) Add git file
4dbcd3c Revert "added wrong file"
4446923 added wrong file
b2d3e08 resolved merge conflict
a43c2be update sample in branch2
f5b983c update sample in branch1
6fe33ae initial commit with sample
6f26c97 add sample.txt in branch1

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (detached-branch)
$ git log --oneline -1
3ab3de9 (HEAD -> detached-branch) Add git file

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (detached-branch)
$
```

step 6: (Optional) Push the New Branch to GitHub

$ git push -u origin detached-branch

```
brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (detached-branch)
$ git push -u origin detached-branch
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0
remote:
remote: Create a pull request for 'detached-branch' on GitHub by visiting:
remote:      https://github.com/rajkumari-hub/Git_Challenge/pull/new/detached-branch
remote:
To github.com:rajkumari-hub/Git_Challenge.git
 * [new branch]      detached-branch -> detached-branch
branch 'detached-branch' set up to track 'origin/detached-branch'.

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (detached-branch)
$
```



11.Git Hooks Challenge

- Configure apre-commithook to reject commits without a message format (e.g., must start withJIRA-XXX).

Step1: Go to the Hooks Directory

$ cd .git.hooks

```
rajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ ls -a
./  ../  .git/  branch.txt  cherry.txt  college  git.txt  reset.txt  rules  sample  student

rajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ cd .git

rajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge/.git (GIT_DIR!)
$ ls
COMMIT_EDITMSG  ORIG_HEAD    config       hooks/  info/  objects/     refs/
HEAD            REBASE_HEAD  description  index   logs/  packed-refs

rajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge/.git (GIT_DIR!)
$ cd hooks

rajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge/.git/hooks (GIT_DIR!)
$
```

Step 2: Create the commit-msg Hook

vi commit-msg

Add the following Script and give permissions

$ chmod +x commit-msg

```
brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge/.git/hooks (GIT_DIR!)
$ vi commit-msg

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge/.git/hooks (GIT_DIR!)
$ chmod +x commit-msg
```

```
#!/bin/sh

commit_msg=$(cat "$1")

if ! echo "$commit_msg" | grep -qE '^JIRA-[0-9]+'; then
    echo "ERROR: Commit message must start with JIRA-XXX"
    echo "Example: JIRA-101 Add login feature"
    exit 1
fi

exit 0
```

Step 3: Return to the Repository

cd ../..

Step 4: Create a Sample File

$ echo "Git Hooks Example" > hooks.txt

Step 5: Stage the File

$ git add hooks.txt

Step 6: Test an Invalid Commit Message

$ git commit -m "Added hooks file"

```
brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge/.git/hooks (GIT_DIR!)
$ cd ../..

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ echo "Git Hooks Example" > hooks.txt

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ git add hooks.txt
warning: in the working copy of 'hooks.txt', LF will be replaced by CRLF the next time Git touches
 it

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ git commit -m "Added hooks file"
ERROR: Commit message must start with JIRA-XXX
Example: JIRA-101 Add login feature

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$
```

Step 7: Test a Valid Commit Message

$ git commit -m "JIRA-101 Added hooks file"

Step 8: Verify the Commit

$ git log --oneline

```
brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ git commit -m "JIRA-101 Added hooks file"
[main 2b2db7e] JIRA-101 Added hooks file
 1 file changed, 1 insertion(+)
 create mode 100644 hooks.txt

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ git log --oneline
2b2db7e (HEAD -> main) JIRA-101 Added hooks file
1baf530 Revert "Added revert example"
8cf090f Added revert example
ef412d9 (tag: v1.0) add student college rules files in one commit
e2dd6d3 Added cherry pick
3ab3de9 (origin/detached-branch, detached-branch) Add git file
4dbcd3c Revert "added wrong file"
4446923 added wrong file
b2d3e08 resolved merge conflict
a43c2be update sample in branch2
f5b983c update sample in branch1
6fe33ae initial commit with sample
6f26c97 add sample.txt in branch1

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$
```

12. Squash Merge vs Rebase Merge

- Show the difference between squash merge and rebase merge with evidence.

Step1: Change the branch main to feature

$ git checkout feature

Step2: Create two files with same name and different content

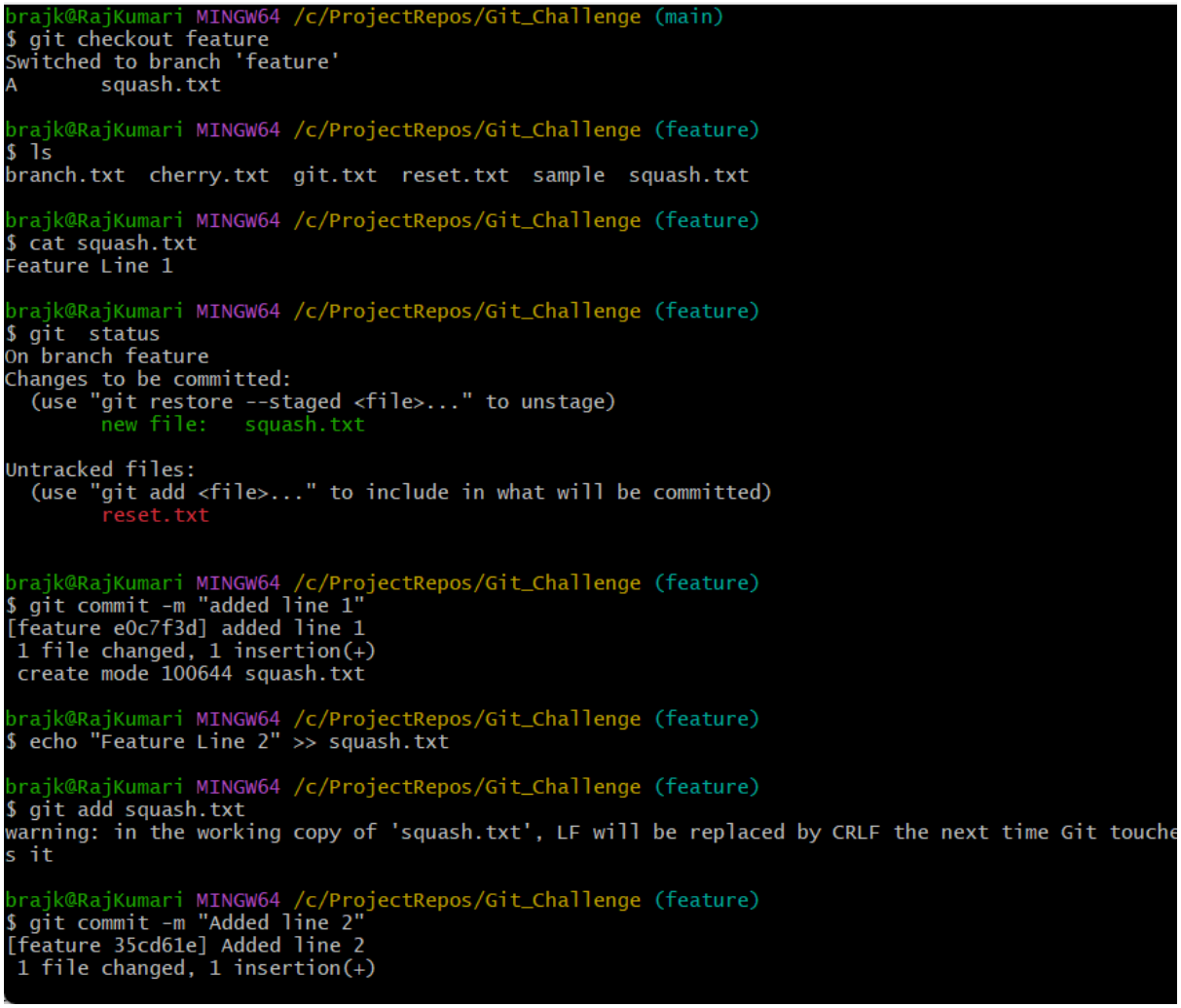
$ vi squash.txt

$ vi squash.txt

Step3: Add and commit

$ git add .

$ git commit -m “added line”



Step 4: View the Commit History

$ git log --oneline

Step 5: Switch Back to Main

$ git checkout main

Step 6: Perform Squash Merge

$ git merge --squash feature

Step 7: Commit the Squashed Changes

$ git commit -m "Squash merged feature-squash"

Step 8: Verify the History (Evidence)

$ git log --oneline

```
e0c7f3d added line 1
ab6a7fc Added cherry pick
3ab3de9 (origin/detached-branch, detached-branch) Add git file
4dbcd3c Revert "added wrong file"
4446923 added wrong file
b2d3e08 resolved merge conflict
a43c2be update sample in branch2
f5b983c update sample in branch1
6fe33ae initial commit with sample
6f26c97 add sample.txt in branch1

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (feature)
$ git checkout main
Switched to branch 'main'
Your branch and 'origin/main' have diverged,
and have 3 and 1 different commits each, respectively.
  (use "git pull" to merge the remote branch into yours)

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ git merge --squash feature
Automatic merge went well; stopped before committing as requested
Squash commit -- not updating HEAD

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ git commit -m "Squash merged feature-squash"
[main c5d556d] Squash merged feature-squash
 1 file changed, 2 insertions(+)
 create mode 100644 squash.txt

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ git commit -m "Squash merged feature-squash"
On branch main
Your branch and 'origin/main' have diverged,
and have 4 and 1 different commits each, respectively.
  (use "git pull" to merge the remote branch into yours)

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        reset.txt

nothing added to commit but untracked files present (use "git add" to track)

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$
```

Part 2: Rebase Merge

Step 1: Create Another Branch

$ git checkout -b feature-rebase

Step 2: Create the First Commit

$ echo "Rebase Line 1" > rebase.txt

$ git add rebase.txt

$ git commit -m "Added rebase line 1"

Step 3: Create the Second Commit

$ echo "Rebase Line 2" >> rebase.txt

git add rebase.txt

$ git commit -m "Added rebase line 2"

Step 4: Rebase the Feature Branch

$ git rebase main

Step 5: Switch to Main

$ git checkout main

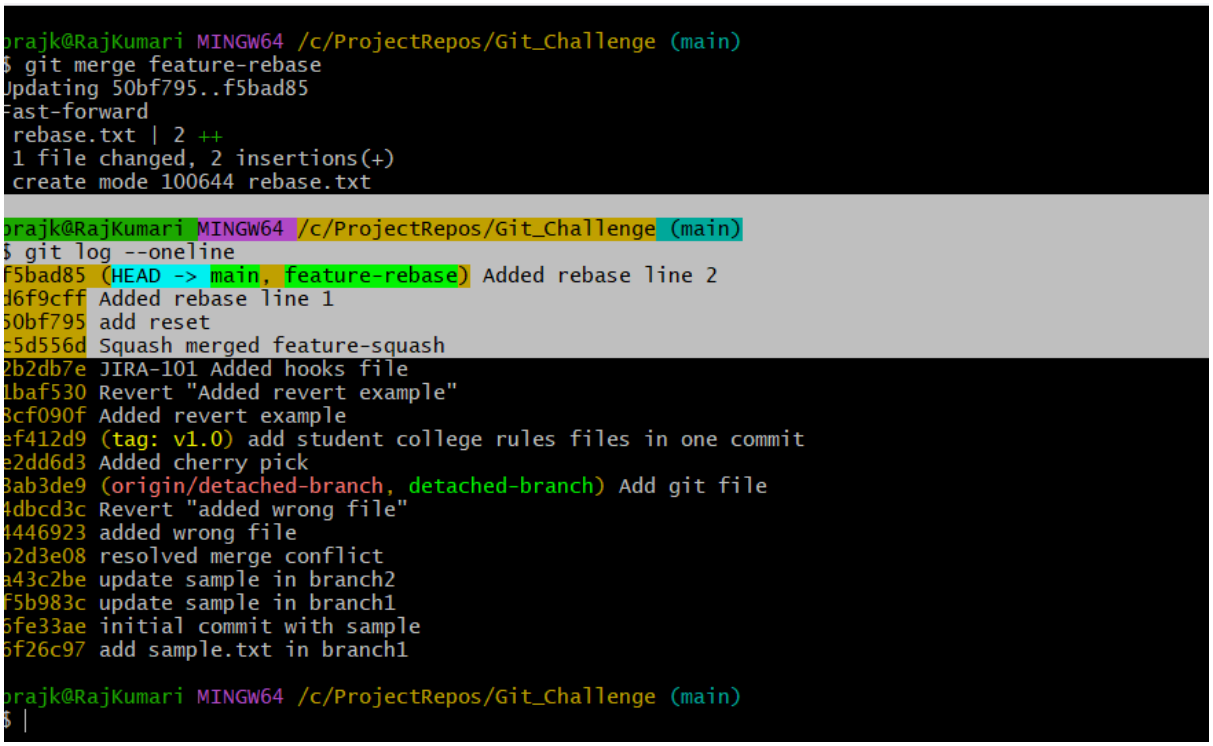
```
brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ git checkout -b feature-rebase
Switched to a new branch 'feature-rebase'

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (feature-rebase)
$ echo "Rebase Line 1" > rebase.txt

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (feature-rebase)
$ git add rebase.txt
warning: in the working copy of 'rebase.txt', LF will be replaced by CRLF the next time Git touche
s it

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (feature-rebase)
$ git commit -m "Added rebase line 1"
[feature-rebase d6f9cff] Added rebase line 1
 1 file changed, 1 insertion(+)
 create mode 100644 rebase.txt

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (feature-rebase)
$ echo "Rebase Line 2" >> rebase.txt

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (feature-rebase)
$ git add rebase.txt
warning: in the working copy of 'rebase.txt', LF will be replaced by CRLF the next time Git touche
s it

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (feature-rebase)
$ git commit -m "Added rebase line 2"
[feature-rebase f5bad85] Added rebase line 2
 1 file changed, 1 insertion(+)

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (feature-rebase)
$ git rebase main
Current branch feature-rebase is up to date.

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (feature-rebase)
$ git checkout main
Switched to branch 'main'
Your branch and 'origin/main' have diverged,
and have 5 and 1 different commits each, respectively.
  (use "git pull" to merge the remote branch into yours)

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$
```

Step6: Merge the Rebased Branch

$ git merge feature-rebase

Step 7: Verify the History (Evidence)

$ git log –oneline



13. Fork & Pull Request Workflow

- Fork a repo, make a change, and submit a pull request to the original repo.

Objective: Fork a GitHub repository, make a change in your fork, and submit a Pull Request (PR) to the original repository.

Step 1: Open the Original Repository

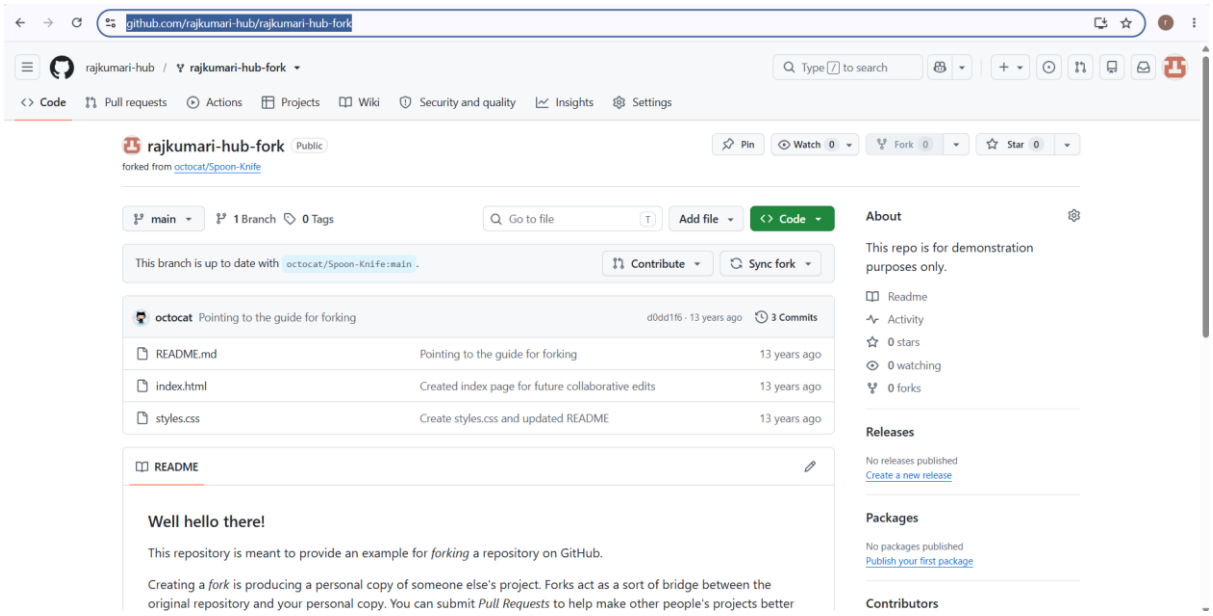
1. Sign in to GitHub.
2. Open the original repository.

Step 2: Fork the Repository

1. Click the Fork button.

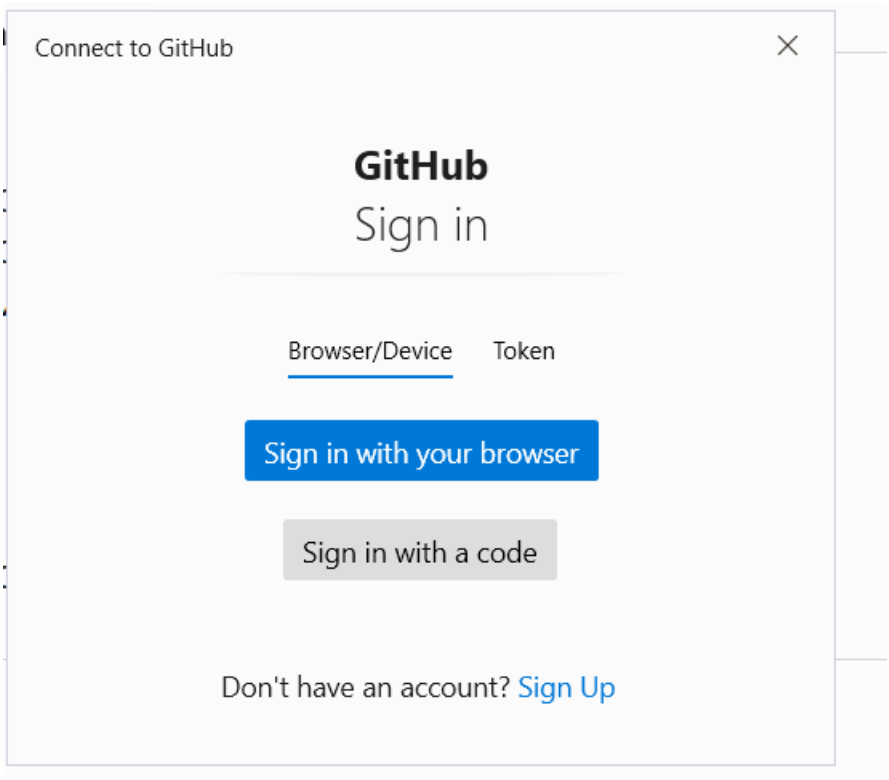
2. Select your GitHub account.
3. GitHub creates a copy of the repository under your account.

Forked Repository: https://github.com/rajkumari-hub/rajkumari-hub-fork

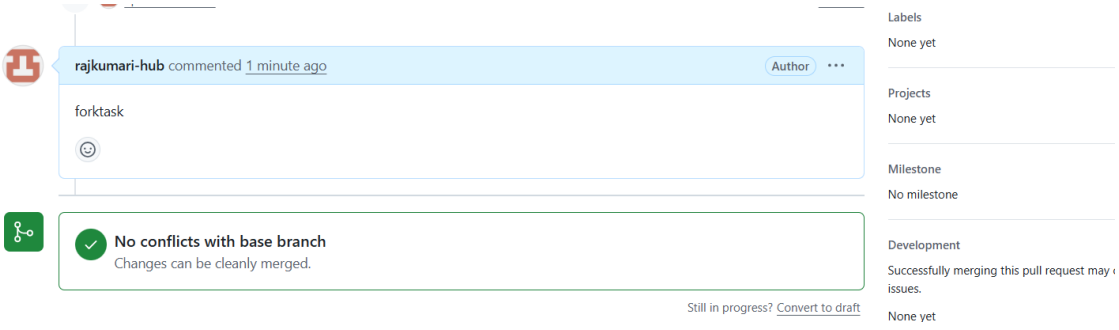


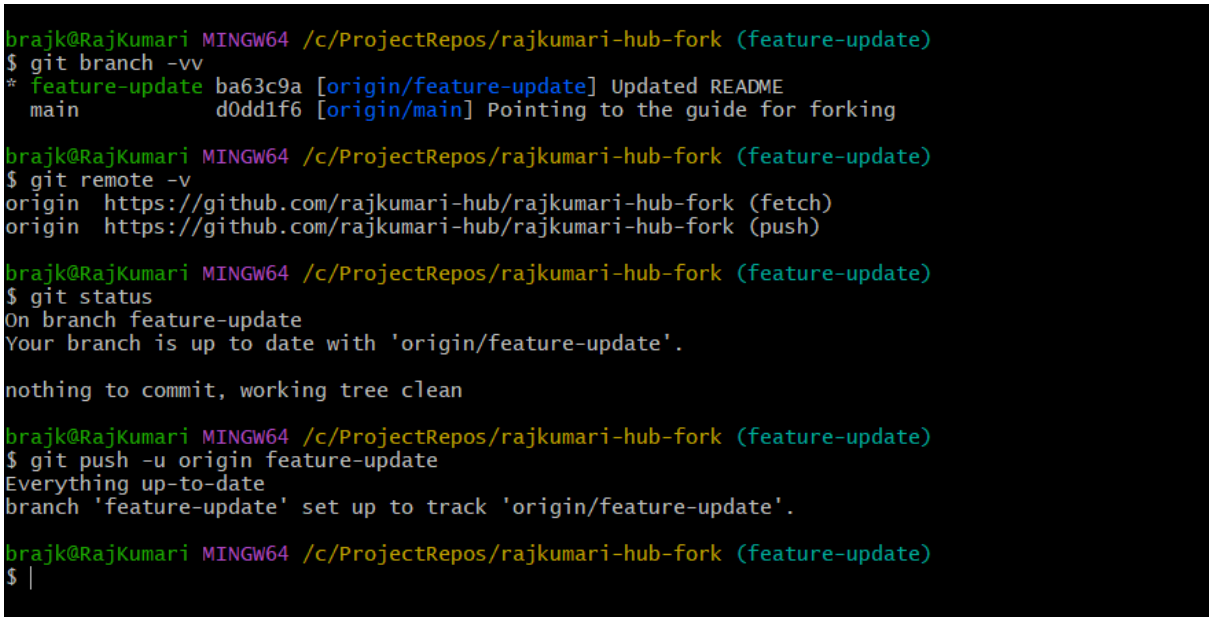
```
brajk@RajKumari MINGW64 /c/ProjectRepos
$ pwd
/c/ProjectRepos

brajk@RajKumari MINGW64 /c/ProjectRepos
$ git clone https://github.com/rajkumari-hub/rajkumari-hub-fork
Cloning into 'rajkumari-hub-fork'...
remote: Enumerating objects: 10, done.
remote: Total 10 (delta 0), reused 0 (delta 0), pack-reused 10 (from 1)
Receiving objects: 100% (10/10), done.
Resolving deltas: 100% (1/1), done.

brajk@RajKumari MINGW64 /c/ProjectRepos
$ ls
Git_Challenge/  SparseRepo/  rajkumari-hub-fork/

brajk@RajKumari MINGW64 /c/ProjectRepos
$ cd rajkumari-hub-fork

brajk@RajKumari MINGW64 /c/ProjectRepos/rajkumari-hub-fork (main)
$ git checkout -b feature-update
Switched to a new branch 'feature-update'

brajk@RajKumari MINGW64 /c/ProjectRepos/rajkumari-hub-fork (feature-update)
$ echo "Fork and Pull Request Demo" >> README.md

brajk@RajKumari MINGW64 /c/ProjectRepos/rajkumari-hub-fork (feature-update)
$ git status
On branch feature-update
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")

brajk@RajKumari MINGW64 /c/ProjectRepos/rajkumari-hub-fork (feature-update)
$
```

Here, we need to give github credentials




```
brajk@RajKumari MINGW64 /c/ProjectRepos/rajkumari-hub-fork (feature-update)
$ git push -u origin feature-update
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 373 bytes | 373.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'feature-update' on GitHub by visiting:
remote:      https://github.com/rajkumari-hub/rajkumari-hub-fork/pull/new/feature-update
remote:
To https://github.com/rajkumari-hub/rajkumari-hub-fork
 * [new branch]      feature-update -> feature-update
branch 'feature-update' set up to track 'origin/feature-update'.

brajk@RajKumari MINGW64 /c/ProjectRepos/rajkumari-hub-fork (feature-update)
$
```





14. Recover Lost Commit

- Commit something, reset hard, and then recover it using git reflog.

Step1: Make sure you are on the main branch

$ git checkout main

Step 2: Create a new file

$ echo "Recover lost commit example" > recover.txt

Step 3: Stage the file

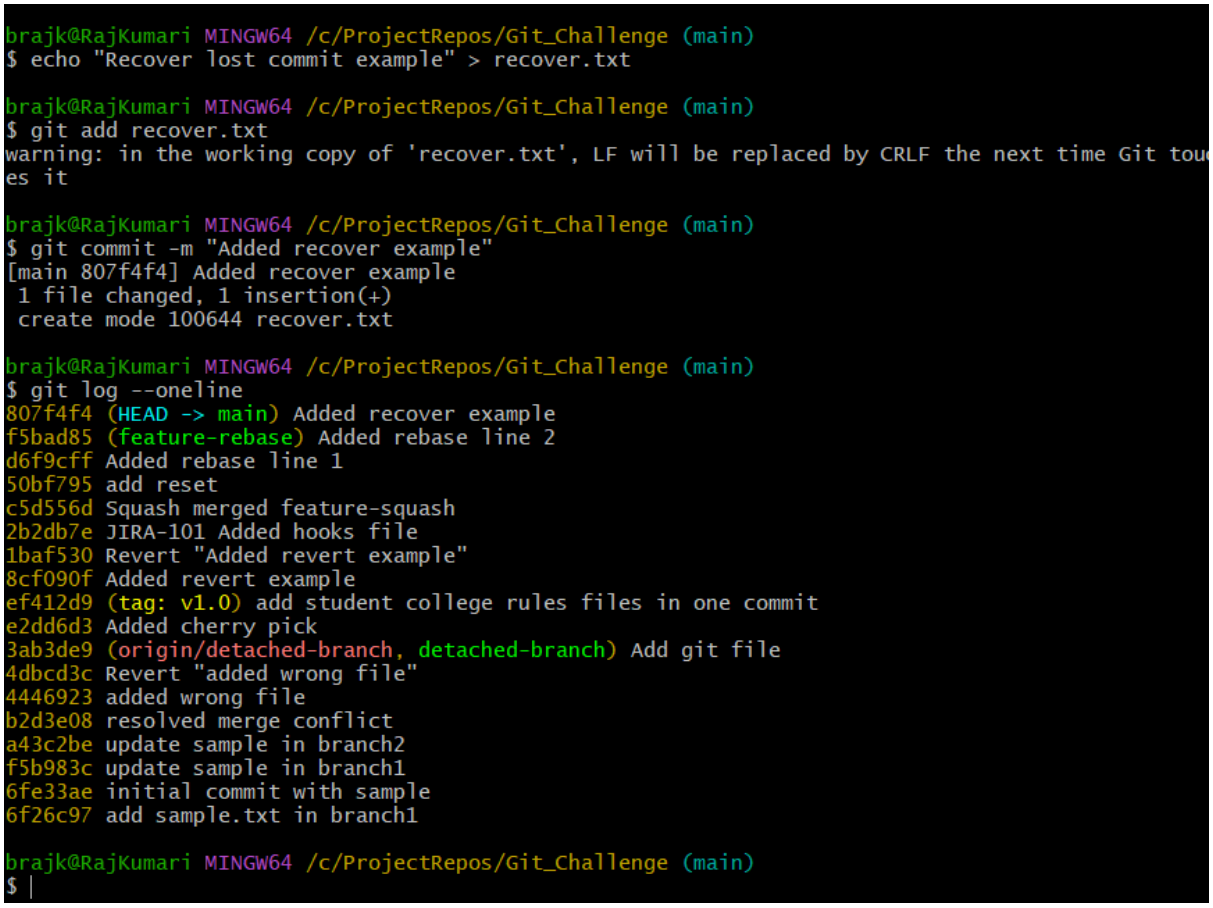
$ git add recover.txt

Step 4: Commit the file

$ git commit -m "Added recover example"

Step 5: View the commit history

$ git log --oneline



Step6: Remove the commit using hard reset

Move HEAD to the previous commit.

$ git reset --hard HEAD~1

The commit "**Added recover example**" is no longer visible in git log.

Step 7: Verify the commit is gone

$ git log --oneline

Step 8: View the reflog

$ git rflog

```
brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ git reset --hard HEAD~1
HEAD is now at f5bad85 Added rebase line 2

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ git log --oneline
f5bad85 (HEAD -> main, feature-rebase) Added rebase line 2
d6f9cff Added rebase line 1
50bf795 add reset
c5d556d Squash merged feature-squash
2b2db7e JIRA-101 Added hooks file
1baf530 Revert "Added revert example"
8cf090f Added revert example
ef412d9 (tag: v1.0) add student college rules files in one commit
e2dd6d3 Added cherry pick
3ab3de9 (origin/detached-branch, detached-branch) Add git file
4dbcd3c Revert "added wrong file"
4446923 added wrong file
b2d3e08 resolved merge conflict
a43c2be update sample in branch2
f5b983c update sample in branch1
6fe33ae initial commit with sample
6f26c97 add sample.txt in branch1

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ git reflog
f5bad85 (HEAD -> main, feature-rebase) HEAD@{0}: reset: moving to HEAD~1
807f4f4 HEAD@{1}: commit: Added recover example
f5bad85 (HEAD -> main, feature-rebase) HEAD@{2}: merge feature-rebase: Fast-forward
50bf795 HEAD@{3}: checkout: moving from feature-rebase to main
f5bad85 (HEAD -> main, feature-rebase) HEAD@{4}: commit: Added rebase line 2
d6f9cff HEAD@{5}: commit: Added rebase line 1
50bf795 HEAD@{6}: checkout: moving from main to feature-rebase
50bf795 HEAD@{7}: commit: add reset
c5d556d HEAD@{8}: commit: Squash merged feature-squash
2b2db7e HEAD@{9}: checkout: moving from feature to main
35cd61e (feature) HEAD@{10}: commit: Added line 2
e0c7f3d HEAD@{11}: commit: added line 1
ab6a7fc HEAD@{12}: checkout: moving from main to feature
2b2db7e HEAD@{13}: checkout: moving from feature to main
ab6a7fc HEAD@{14}: checkout: moving from main to feature
2b2db7e HEAD@{15}: commit: JIRA-101 Added hooks file
1baf530 HEAD@{16}: checkout: moving from detached-branch to main
```

Step 9: Recover the lost commit

git reset --hard 807f4f4

Replace 807f4f4 with the commit hash shown in your reflog.

Step 10: Verify recovery

$ git log --oneline

Step 11: Verify the file exists

$ ls

$ cat recover.txt

```
brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ git reset --hard 807f4f4
HEAD is now at 807f4f4 Added recover example

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ git log --oneline
807f4f4 (HEAD -> main) Added recover example
f5bad85 (feature-rebase) Added rebase line 2
d6f9cff Added rebase line 1
50bf795 add reset
c5d556d Squash merged feature-squash
2b2db7e JIRA-101 Added hooks file
1baf530 Revert "Added revert example"
8cf090f Added revert example
ef412d9 (tag: v1.0) add student college rules files in one commit
e2dd6d3 Added cherry pick
3ab3de9 (origin/detached-branch, detached-branch) Add git file
4dbcd3c Revert "added wrong file"
4446923 added wrong file
b2d3e08 resolved merge conflict
a43c2be update sample in branch2
f5b983c update sample in branch1
6fe33ae initial commit with sample
6f26c97 add sample.txt in branch1

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ ls
branch.txt  college  hooks.txt   recover.txt  rules   squash.txt
cherry.txt  git.txt  rebase.txt  reset.txt    sample  student

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$ cat recover.txt
Recover lost commit example

brajk@RajKumari MINGW64 /c/ProjectRepos/Git_Challenge (main)
$
```